The road to import boost

A library developer's journey into C++20 modules

Rubén Pérez Hidalgo <rubenperez038@gmail.com>
The C++ Alliance

using std::cpp 2026

About me

Who I am

- ✓ Boost.MySQL author
- ✓ Boost.Redis co-maintainer
- ✓ Initiative: support import Boost

Who I am not

- × Involved in module standardization
- × Compiler developer
- × Build system developer

I am just a user

(that has read the standard)

Schedule

1. Developing **applications** using modules

Servertech Chat: 4k LOC

based on Boost.Asio

2. Supporting modules in **libraries**

import boost

Why modules?

- ✓ Reduce **build times**
- ✓ Better **isolation**
- ✓ Prevent **ODR violations**

```cpp
// I hope you don't mind me pouring a couple
// of entities into your translation unit
#include <boost/asio.hpp>

namespace mylib {

// Please don't use :)
namespace detail {
class connection_impl {};
}

// I hope you're not ever mixing standard levels
#ifdef __cpp_lib_string_view
void print(std::string_view sv);
#else
void print(const std::string& s);
#endif

}
```

Importing modules

```cpp
import std;

int main()
{
    std::println("Hello world!");
}
```

```cmake
add_executable(main import_std.cpp)
target_compile_features(main PUBLIC cxx_std_23)
set_target_properties(main PROPERTIES CXX_MODULE_STD ON)
```

```
cmake -DCMAKE_EXPERIMENTAL_CXX_IMPORT_STD=d0edc3af-4c50-42ea-a356-e2862fe7a444 ...
```

```
~/00-import-std/import_std.cpp:1:8: fatal error: module 'std' not found
    1 | import std;
      | ~~~~~~~^~~
1 error generated.
ninja: build stopped: subcommand failed.
```

The std module needs to be **built by us**

Primary module interface

```cpp
// File: base64.hpp
#include <string>
#include <cstddef>
#include <span>

namespace chat {
inline std::string
base64_encode(std::span<const std::byte> value)
{
   // ...
}
} // namespace chat

// File: main.cpp
#include "base64.hpp"
#include <iostream>

int main()
{
   std::byte arr [10] {};
   std::cout << chat::base64_encode(arr) << '\n';
}
```

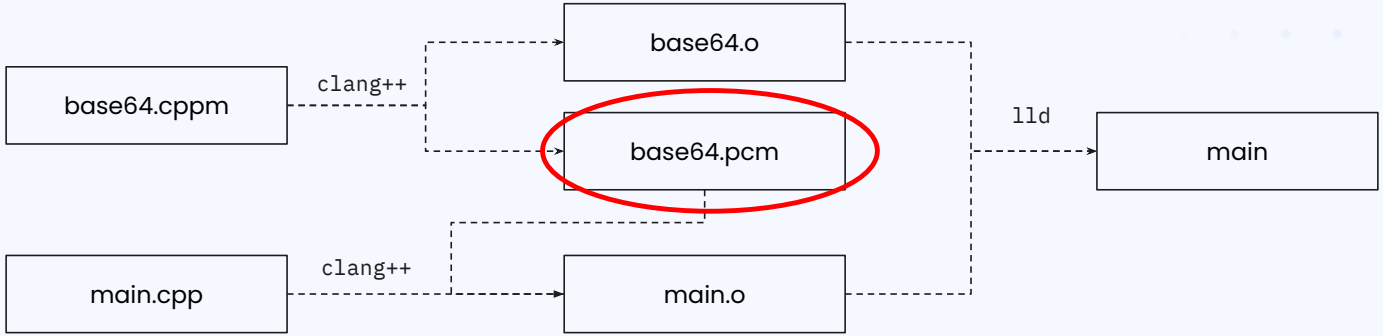
```cpp
// File: base64.cppm
export module base64;

import std;

namespace chat {
export inline std::string
base64_encode(std::span<const std::byte> value)
{
   // ...
}
} // namespace chat

// File: main.cpp
import base64;
import std;

int main()
{
   std::byte arr [10] {};
   std::cout << chat::base64_encode(arr) << '\n';
}
```

Building modules

```
add_library(base64)
target_sources(base64 PUBLIC FILE_SET CXX_MODULES FILES
   base64.cppm)
set_target_properties(base64 PROPERTIES CXX_MODULE_STD ON)
target_compile_features(base64 PUBLIC cxx_std_23)

add_executable(main main.cpp)
target_link_libraries(main PRIVATE base64)
```

Importable TUs need this syntax

Binary module interface (BMI)



- ✓ Used by the compiler when it sees an import
- ✓ Narrow compatibility (like a precompiled header) ⇒ don't distribute BMIs!
- ✓ Compilation must be performed in order

```
[4/14   7% :: 0.052] Scanning ~/project/base64.cppm for CXX dependencies
[4/14  14% :: 0.052] Scanning ~/project/main.cpp for CXX dependencies
[7/14  42% :: 0.184] Generating CXX dyndep file CMakeFiles/base64.dir/CXX.dd
[8/14  50% :: 0.190] Generating CXX dyndep file CMakeFiles/main.dir/CXX.dd
```

Reachability and visibility

```cpp
// File: base64.cppm
export module base64;
import std;

namespace chat {
inline std::size_t encoded_size(std::size_t data_size)
{
    return 4 * ((n + 2) / 3);
}

export inline std::string
base64_encode(std::span<const std::byte> value)
{
   std::string res (encoded_size(value.size()));
   // ...
}
} // namespace chat
```

```cpp
// File: main.cpp
import base64;

int main()
{
   std::byte arr [10] {};
   // std::cout << chat::encoded_size(10) << '\n';
   std::cout << chat::base64_encode(arr) << '\n';
}
```

Reachable, but not visible

Visible and reachable

Visible: available to us using name lookup

Reachable: semantic properties available to the compiler

Separate compilation

```cpp
// File: base64.hpp
#include <string>
#include <cstddef>
#include <span>

namespace chat {
std::string base64_encode(std::span<const std::byte> value);
} // namespace chat

// File: base64.cpp
#include "base64.hpp"

std::string
chat::base64_encode(std::span<const std::byte> value)
{
   // ...
}
```

```cpp
// File: base64.cppm
export module base64;

import std;

namespace chat {
export std::string
base64_encode(std::span<const std::byte> value)
{
   // ...
}
} // namespace chat
```

`base64.cppm` is a **translation unit**!

Problem: rebuilds

Implementation units

```cpp
// File: base64.hpp
#include <string>
#include <cstddef>
#include <span>

namespace chat {
std::string base64_encode(std::span<const std::byte> value);
} // namespace chat

// File: base64.cpp
#include "base64.hpp"

std::string
chat::base64_encode(std::span<const std::byte> value)
{
   // ...
}
```

```cpp
// File: base64.cppm
export module base64;
import std;

namespace chat {
export std::string
base64_encode(std::span<const std::byte> value);
} // namespace chat

// File: base64.cpp (implementation unit)
module base64;
import std;

std::string
chat::base64_encode(std::span<const std::byte> value)
{
   // ...
}
```

```cmake
add_library(base64
   base64.cpp)
target_sources(base64 PUBLIC FILE_SET CXX_MODULES FILES
   base64.cppm)
```

Partition units

```cpp
// File: base64.cppm (implementation partition unit)
module server:base64;
import std;

namespace chat {
std::string base64_encode(std::span<const std::byte> value);
}

// File: base64.cpp (implementation unit)
module server;
import :base64;

std::string chat::base64_encode(std::span<const std::byte> value)
{
   // ...
}
```

```cmake
add_library(server
   base64.cpp server.cpp)
target_sources(server PUBLIC FILE_SET CXX_MODULES FILES
   base64.cppm server.cppm)
```

```cpp
// File: server.cppm (primary interface unit)
export module server;

namespace chat {
export void serve_forever(unsigned short port);
}

// File: server.cpp (implementation unit)
module server;

import std;
import :base64;

void chat::serve_forever(unsigned short port)
{
   // Use base64_encode here
}
```

Module attachment

```cpp
// File: server.cppm
export module server;

namespace chat {
export void serve_forever(unsigned short port);
}

// File: main.cpp
import server;

void chat::serve_forever(unsigned short port) { /* ... */ }

int main()
{
   chat::serve_forever(80);
}
```

Becomes **attached** to module `server`

Must be defined in module `server`

```
~/project/main.cpp:3:12: error: declaration of 'serve_forever' in the global module follows declaration in module server
    3 | void chat::serve_forever(unsigned short port) {}
      |            ^
~/project/server.cppm:6:13: note: previous declaration is here
    6 | export void serve_forever(unsigned short port);
      |
```

Macros

```cpp
// File: base64.cpp
module server;
import :base64;
import std;

static std::string base64_encode_long(std::span<const std::byte> value)
{
   assert(!value.empty());
   // ...
}

static std::string base64_encode_short(std::span<const std::byte> value) { /* ... */ }

std::string chat::base64_encode(std::span<const std::byte> value)
{
   return value.size() >= 64 ? base64_encode_long(value) : base64_encode_short(value);
}
```

```
~/project/base64.cpp:8:5: error: use of undeclared identifier 'assert'
    8 |     assert(!value.empty());
```

Macros

```
// File: base64.cpp
module server;
import :base64;
import std;

#include <cassert>

static std::string base64_encode_long(std::span<const std::byte> value)
{
   assert(!value.empty());
   // ...
}

static std::string base64_encode_short(std::span<const std::byte> value) { /* ... */ }

std::string chat::base64_encode(std::span<const std::byte> value)
{
   return value.size() >= 64 ? base64_encode_long(value) : base64_encode_short(value);
}
```

```
~/project/base64.cpp:6:10: warning: '#include <filename>' attaches the declarations to the named module 'server', which is
not usually intended; consider moving that directive before the module declaration [-Winclude-angled-in-module-purview]
    6 | #include <cassert>
```

Global module fragment

```cpp
// File: base64.cpp
module;

#include <cassert>
```

Global module fragment

```cpp
module server;
import :base64;
import std;

static std::string base64_encode_long(std::span<const std::byte> value)
{
   assert(!value.empty());
   // ...
}

static std::string base64_encode_short(std::span<const std::byte> value) { /* ... */ }

std::string chat::base64_encode(std::span<const std::byte> value)
{
   return value.size() >= 64 ? base64_encode_long(value) : base64_encode_short(value);
}
```

Module purview

Dependencies

```cpp
// File: server.cpp (implementation unit)
module;

#include <boost/asio/io_context.hpp>
#include <boost/asio/ip/tcp.hpp>

module server;

import std;

void chat::serve_forever(unsigned short port)
{
   boost::asio::io_context ctx;
   boost::asio::ip::tcp::acceptor acceptor (ctx);
   // ...
   ctx.run();
}
```

```cpp
// File: redis_client.cppm (implementation partition unit)
module;

#include <boost/asio/awaitable.hpp>

module server:redis_client;

namespace chat {

export class redis_client {
public:
   boost::asio::awaitable<std::int64_t> lookup_user(
       std::string_view cookie
   );
};

}
```

Let’s check…

Build times

Let’s measure

?

Isolation

Explicit exports
No macro leaks

✓

ODR violations

No headers
Attachment

✓

Make your bets

How much did we reduce build times?

a) 66%

b) 33%

c) 0%

d) -66%

Make your bets

How much did we reduce build times?

a) 66%

b) 33%

c) 0%

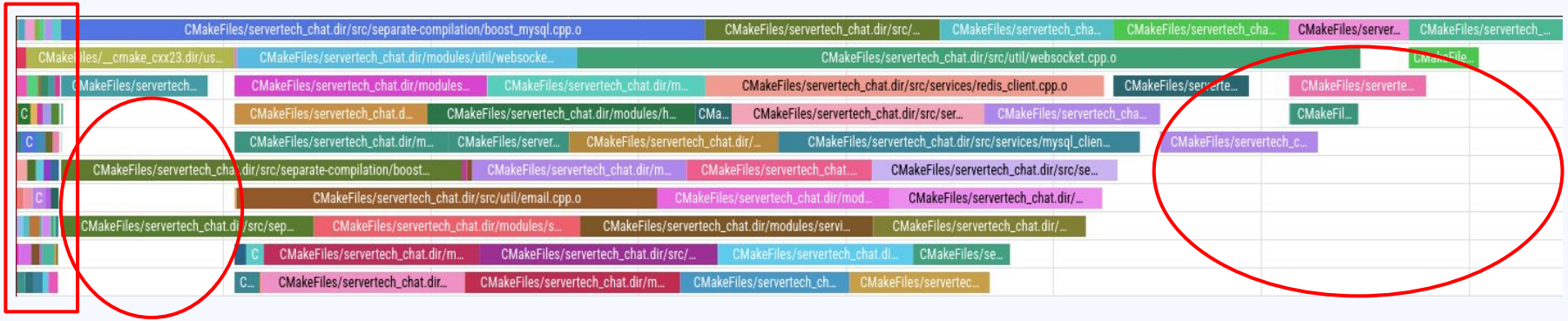
d) -66%

Headers: 45s
Modules: 75s

What went wrong?

Headers

Modules



- Many more TUs
- Equally heavyweight

Because we #include <boost>
Triggers #include<std>

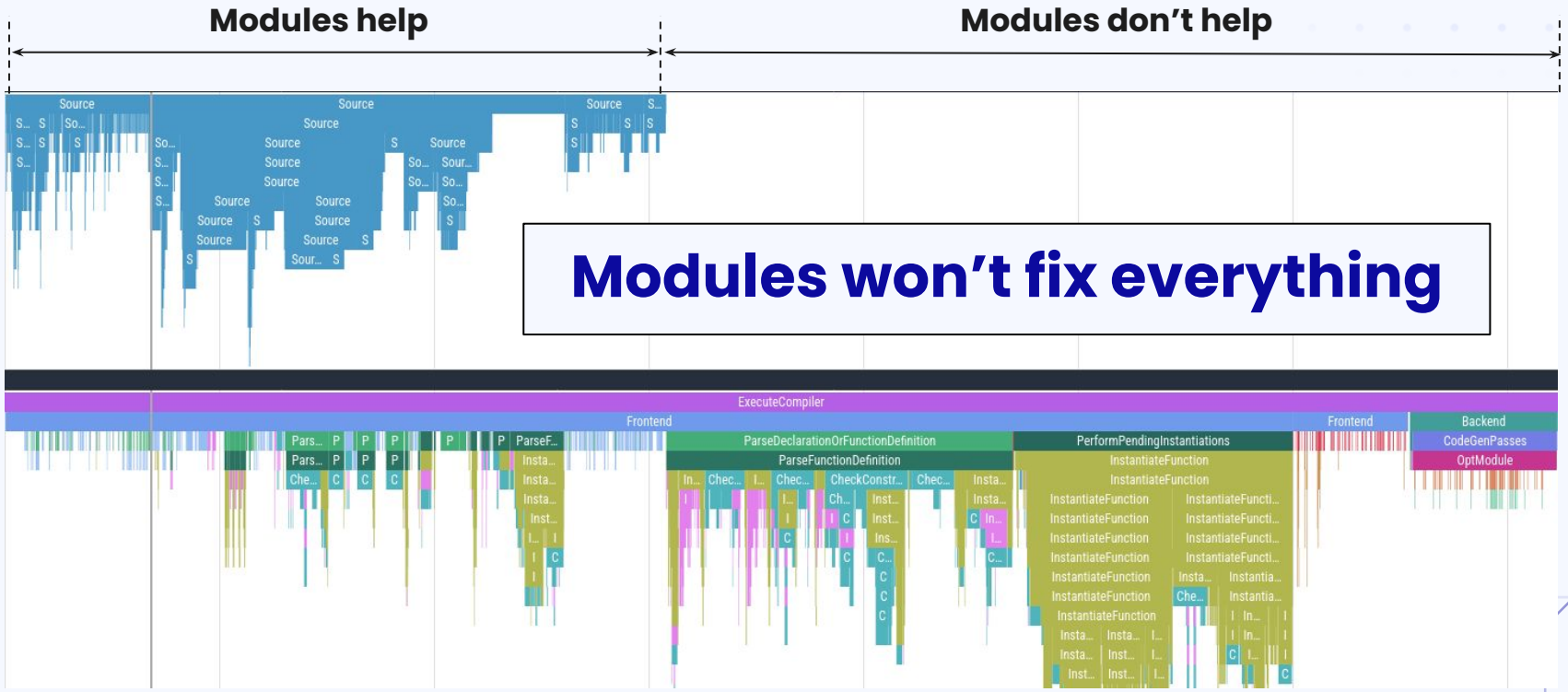
Where do modules help?

```
#include <algorithm>
#include <print>
#include <ranges>
#include <vector>

namespace views = std::views;
namespace ranges = std::ranges;

int main()
{
   std::vector<int> nums = {7, 8, 9, 10, 1, 2, 3, 4, 5, 6};

   auto even_squares = nums
       | views::filter([](int n) { return n % 2 == 0; })
       | views::transform([](int n) { return n * n; })
       | ranges::to<std::vector<int>>();

   ranges::sort(even_squares, std::greater{});
   std::println("Elements: {}", even_squares);
}
```

Headers: 3.6s

Would modules help here?

Where do modules help?



-ftime-trace

import std: 2.5s

Where do modules help?

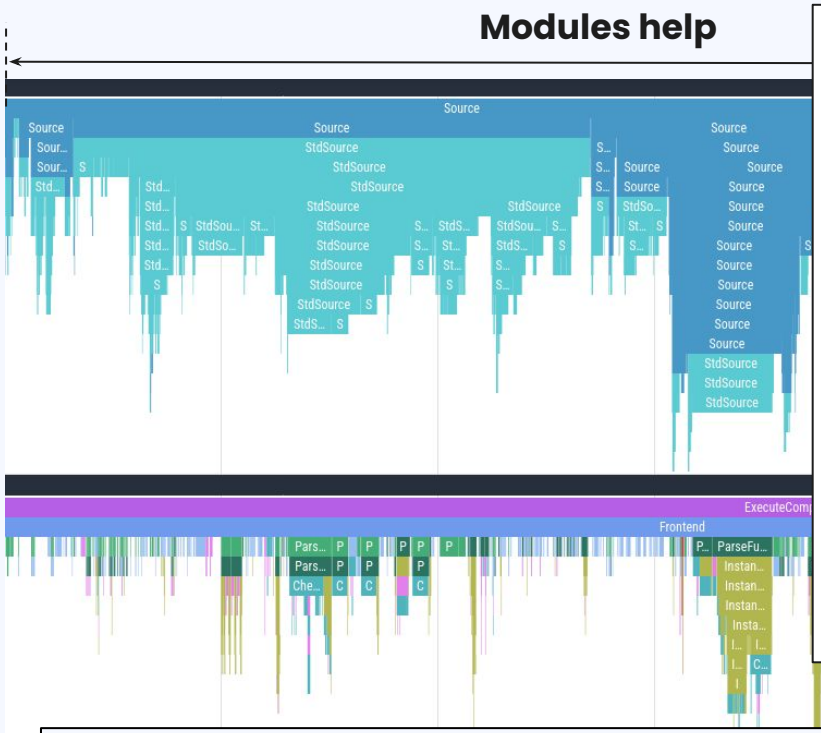
```
#include <boost/asio/io_context.hpp>
#include <boost/asio/ip/address_v4.hpp>
#include <boost/asio/ip/tcp.hpp>
#include <boost/system/error_code.hpp>

#include <iostream>

namespace asio = boost::asio;
using asio::ip::tcp;

int main()
{
   asio::io_context ctx;
   tcp::socket sock(ctx);
   sock.async_connect(
       tcp::endpoint(asio::ip::address_v4::loopback(), 6379),
       [](boost::system::error_code ec) {
           std::cout << "Connect: " << ec << std::endl;
       }
   );
   ctx.run();
}
```

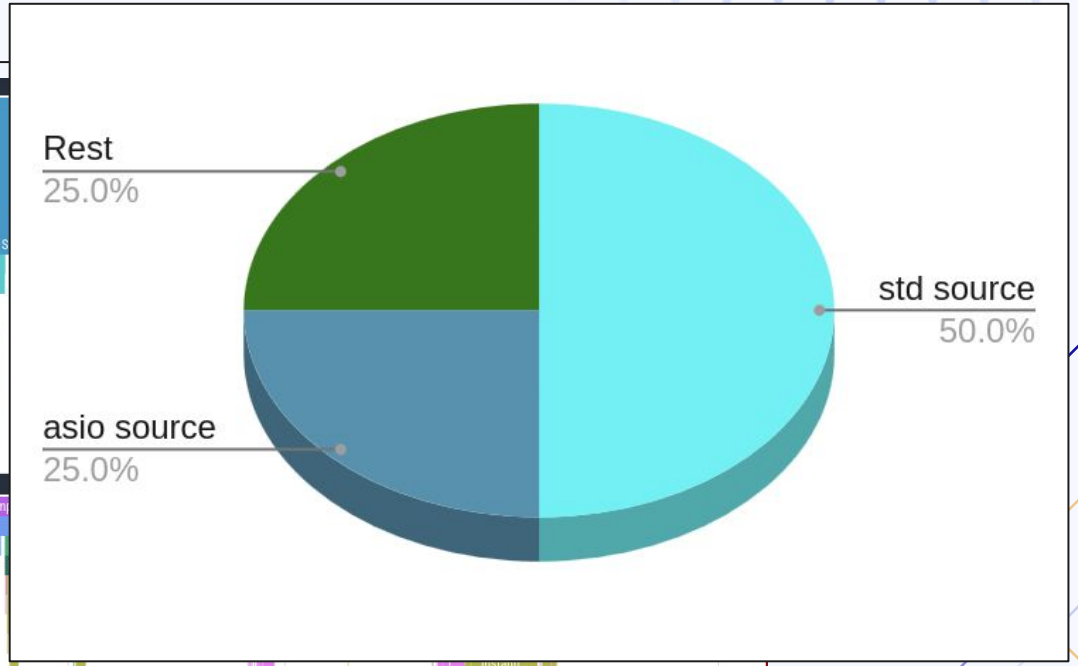
Headers: 3.6s

Would modules help here?

Where do modules help?

Where do modules help?





We need `import std`

Header units

```cpp
// File: redis_client.cppm
module;

import <boost/asio/awaitable.hpp>;

module server:redis_client;

namespace chat {

export class redis_client {
public:
   boost::asio::awaitable<std::int64_t> lookup_user(
       std::string_view cookie
   );
};

}
```

- Unsupported by CMake
- No isolation
- Performance yet to be known

Making import boost work

1. Developing applications using modules

2. Supporting modules in libraries

Can we emulate `import std`, but for Boost?

export using style

```cpp
// included from std.cppm
module;
#include <vector>

export module std;
export namespace std {
 // [vector], class template vector
 using std::vector;

 using std::operator==;
 using std::operator<=>;

 using std::swap;

 // [vector.erasure], erasure
 using std::erase;
 using std::erase_if;

 // ...
}
```

```cpp
// file: boost_asio.cppm
module;

#include <boost/asio/any_io_executor.hpp>
#include <boost/asio/awaitable.hpp>
#include <boost/asio/as_tuple.hpp>
// ...

export module boost.asio;

export namespace boost::asio {

using asio::any_io_executor;
using asio::as_tuple;
using asio::awaitable;
// ...

}
```

export using style

```cpp
// file: boost_asio.cppm
module;

#include <boost/asio/any_io_executor.hpp>
#include <boost/asio/awaitable.hpp>
#include <boost/asio/as_tuple.hpp>
// ...

export module boost.asio;

export namespace boost::asio {

using asio::any_io_executor;
using asio::as_tuple;
using asio::awaitable;
// ...

}
```

```cpp
// file: async_mutex.cppm
module servertech_chat:async_mutex;
import boost.asio;

namespace chat {

// An asynchronous mutex to guarantee mutual exclusion in async code
class async_mutex
{
public:
   // Suspends the current coroutine until
   // the mutex can be acquired, then acquire it
   boost::asio::awaitable<void> lock() { /* ... */ }
};

}  // namespace chat
```

```
~/servertech-chat/server/modules/util/async_mutex.cppm:51:34: error: this function cannot be a coroutine:
'std::__1::coroutine_traits<boost::asio::awaitable<void>, chat::async_mutex &>' has no member named 'promise_type'
   51 |     boost::asio::awaitable<void> lock()
```

GMF discards

```
module;

#include <Windows.h>

export module utils;

import std;

namespace utils {
export std::optional<std::string> read_registry_string(
   const char* sub_key, const char* value_name)
{
   HKEY hkey{};
   LONG result = RegOpenKeyExA(HKEY_CURRENT_USER, sub_key, 0, KEY_READ, &hkey);
   if (result != ERROR_SUCCESS)
       return std::nullopt;

   // ...
}
} // namespace utils
```

Entities in the global module fragment not “used” (**decl-reachable**) by the purview are **discarded**

export using style

```cpp
// file: boost_asio.cppm
module;

#include <boost/asio/any_io_executor.hpp>
#include <boost/asio/awaitable.hpp>
#include <boost/asio/as_tuple.hpp>
// ...

export module boost.asio;

export namespace boost::asio {

using asio::any_io_executor;
using asio::as_tuple;
using asio::awaitable;

inline boost::asio::awaitable<void>
dont_discard_awaitable_coroutine_traits() {
   co_await boost::asio::awaitable<void>{};
}

}
```

Headers: 45s
Modules: 45s

- ✓ Simple
- ✓ Non-intrusive
- ✓ Allows mixing import/include
- ✗ We're including std
- ✗ Doesn't work in the general case

Please **run your entire test suite** using modules before announcing module support in your library!

ABI breaking style

```cpp
// file: modules/boost_asio.cppm
module;

#include <cassert>

export module boost.asio;
import std;

#define BOOST_USE_MODULES
#include <boost/asio.hpp>
```

```cpp
// file: include/boost/asio/detail/config.hpp
#if defined(BOOST_USE_MODULES)
#  define BOOST_ASIO_EXPORT export
#else
#  define BOOST_ASIO_EXPORT
#endif

// file: include/boost/asio/awaitable.hpp
#ifndef BOOST_USE_MODULES
#include <type_traits>
#endif

namespace boost::asio {

BOOST_ASIO_EXPORT
template <typename T, typename Executor = any_io_executor>
class awaitable { /* ... */ };

}
```

Wait ABI what?

```
// File: base64.cppm

module server:base64;
import std;

namespace chat {
std::string base64_encode(std::span<const std::byte> value);
}
```

Becomes **attached** to module server

```
$> nm -C libserver.a

base64.cpp.o:
0000000000000000 T chat::base64_encode@server(std::__1::span<std::byte const, 18446744073709551615ul>)
```

Symbols attached to a named module get a different mangling scheme

ABI breaking style

- ✓ Works on every compiler
- ✓ Best for build times and ODR detection
- ✗ Non-trivial code changes
- ✗ Breaks ABI
- ✗ Imports and includes can’t mix

export extern style

```cpp
// file: modules/boost_asio.cppm
module;

#include <cassert>

export module boost.asio;
import std;

#define BOOST_USE_MODULES
extern "C++" {
#include <boost/asio.hpp>
}
```

Declarations exported, but not attached to `boost.asio`

- ✓ Works on every compiler
- ✓ Doesn't break ABI
- ✓ Allows mixing import/include
- ✗ Non-trivial code changes
- ✗ Slightly worse for ODR violation detections

Modularization styles

	export using	export extern	ABI breaking
Non-intrusive	✓	✗	✗
Works in the general case	✗	✓	✓
Mixing include/import	✓	✓	✗
Keeps ABI stable	✓	✓	✗
Build times	Worst	Better	Best
Improved ODR violation detection	✗	✗	✓
Users	libstdc++, libc++	MSVC STL, fmt	Boost prototype, fmt

import boost prototype

- ABI breaking style
- Bottom up: in dependency order
- Import translation (“compatibility headers”)

```
cmake \
 -G Ninja \
 -DBOOST_USE_MODULES=ON \
 path/to/boost
```

```
#include <boost/mp11/list.hpp>
```

→

```
#if defined(BOOST_USE_MODULES)
import boost.mp11;
#define BOOST_MP11_VERSION 109100
```

It’s just hard…

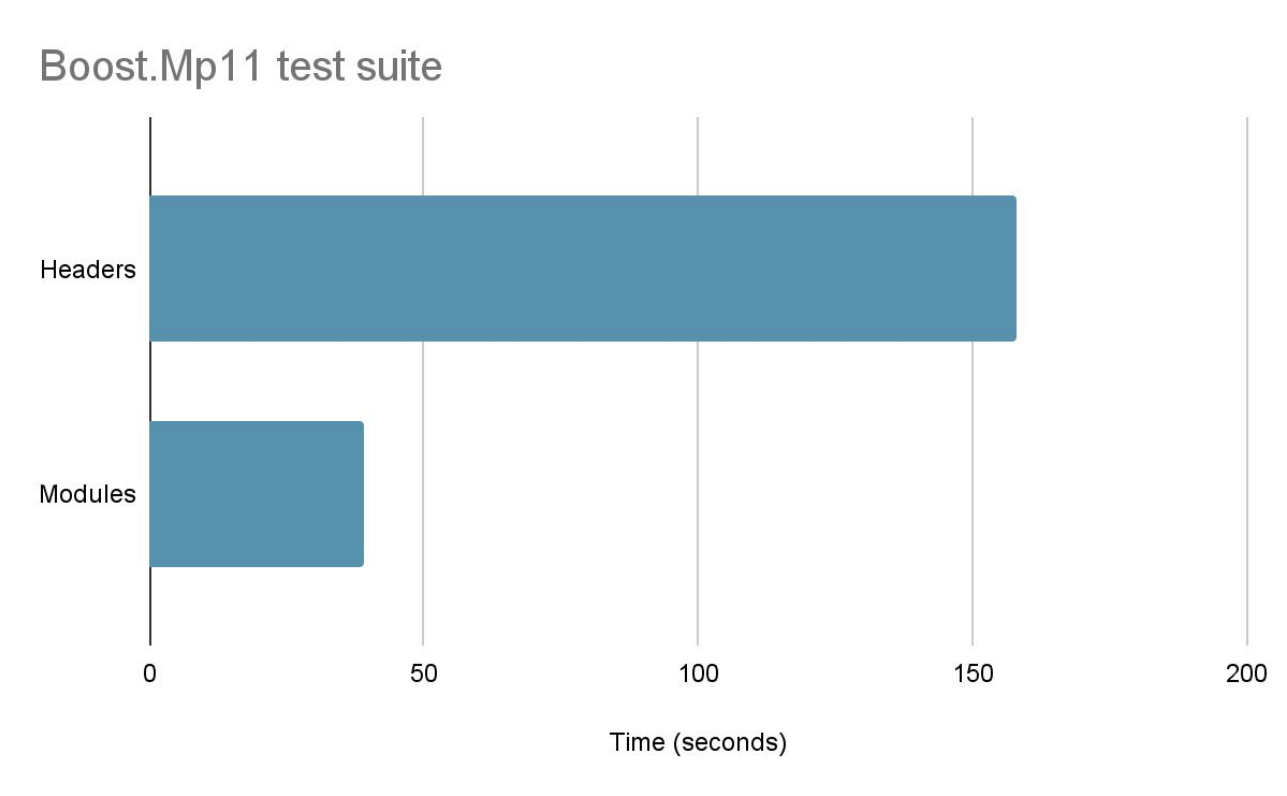
```
#if defined(BOOST_USE_MODULES) && !defined(BOOST_MP11_INTERFACE_UNIT)

#ifndef BOOST_IN_MODULE_PURVIEW
import boost.mp11;
#endif

#else

namespace boost { namespace mp11 { /* contents */ } }

#endif
```

```
module;

#include <math.h>
#include <cerrno>
// others

module boost.charconv;

import std;
import boost.core;

#define BOOST_IN_MODULE_PURVIEW
#include "to_chars.cpp"
```

```
// Make the header safe to include from libraries supporting modules
#if defined(BOOST_IN_MODULE_PURVIEW) && !defined(BOOST_THROW_EXCEPTION_HPP)
#  error "Please #include <boost/throw_exception.hpp> in your GMF"
#endif

#ifndef BOOST_THROW_EXCEPTION_HPP
#define BOOST_THROW_EXCEPTION_HPP

// Can’t be reliably included in purviews
#include <version>
#include <boost/config.hpp>

#if defined( BOOST_NO_EXCEPTIONS )
#define BOOST_THROW_EXCEPTION(x) /* ... */
#else
#define BOOST_THROW_EXCEPTION(x) /* ... */
#endif

#endif
```

```
BOOST_SYSTEM_MODULE_EXPORT class error_code {
public:
   inline char const* category_name() const { return d1_.cat_->name(); }
};
```

Some good news



~230 small TUs

75% improvement

Tooling support

- CMake >= 3.28 (modules), `import std` still experimental
- MSVC >= 14.34 (modules), >= 14.36 (`import std`)
- Clang >= 16 (modules), >= 18 with libc++ (`import std`, experimental)
- GCC >= 14 (modules, experimental), >= 15 with libstdc++ (`import std`, experimental)
- clangd >= 18 (experimental, problems when adding TUs), Visual Studio IntelliSense
- Header units supported in XMake and build2
- **Mixing** import std and includes causes **problems**

My takeaways

- Modules have the potential to be **useful**
- The ecosystem is **still catching up**
- The **transition** mechanism is **not working** yet
- **Modularizing existing libraries** is essential
- "**Modules don't solve everything**"

Thanks!

Do you have any questions?

https://github.com/boostorg/cmake/pull/105

https://github.com/anarthal/boost-cmake/blob/feature/cxx20-modules/modules.md

https://github.com/anarthal/servertech-chat/tree/feature/cxx20-modules

Rubén Pérez Hidalgo
The C++ Alliance

This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**